

生成式软件开发 (7)

– Agent中的工具

大纲

01 **为什么需要工具**

大模型的局限性 • Agent的定义 • 具体场景对比

02 **Agent 中如何引入工具**

自主决定模式 • 强制工作流模式 • 两种模式的对比

03 **工具的两种主要形态**

CLI工具 • API工具 • 形态对比与适用场景

04 **CLI 工具详解**

技术定义 • 执行流程 • CLI vs API 对比

05 **Demo 演示**

批阅作业并上传至飞书文档的完整工作流

06 **边界与安全**

工具分类 • Human-in-the-loop • 安全框架

01

为什么需要工具

大模型的局限性 • Agent的定义 • 具体场景对比

为什么需要工具：大模型的局限性

知识有截止日期

大模型中的知识有**截止日期**，就像一本只写到2025年的百科全书。

它无法获取实时信息，不知道今天的新闻、当前的股价、最新的技术文档。工具让它能够搜索网页、查询数据库、访问API，突破知识的时间边界。

无法主动行动

大模型**只会说话，没有手脚**。它可以给你建议、解释概念、生成文本，但无法真正"做事"。

工具赋予它行动力：创建文件、执行命令、调用API、操作数据库。让它从"思考者"变成"行动者"。

Agent的定义

LLM（推理引擎）
+ Planning（任务分解与策略规划）
+ Session/Context/Skills/Memory（上下文与记忆管理）
+ **Tools（工具调用）**

= Agent

其中， **工具是重要的一环** 。工具让大模型具备了与外部世界交互的能力，是实现从"思考"到"行动"的关键。



具体场景

具体场景对比

场景	没有工具的大模型	有工具的 Agent
写代码	给你代码文本	直接创建文件、运行测试、修复报错
数据分析	解释分析方法	读取你的Excel，画图，输出报告
旅行规划	推荐景点	查实时机票、酒店价格，生成行程表
学习辅助	讲解概念	根据你的作业自动出题、批改、讲解

注意： 工具不是让大模型变聪明，而是让它从"**知道**"变成"**做到**"。工具是连接"智能"与"世界"的桥梁。

02

Agent 中如何引入工具

自主决定模式 • 强制工作流模式 • 两种模式的对比

Agent 中引入工具的两种模式

01 大模型自主决定

Autonomous Routing

代表机制

全局聊天框 + MCP 扩展的工具

工作方式

我们把所有工具的“说明书”都给大模型，只提最终目标（如“帮我修个Bug”）。
大模型自己推理该用搜索、还是用 Terminal。

特点

像雇了一个"高级专家"，极其灵活，适合解决未知和开放性问题，但也容易跑偏或出错。

Agent 中引入工具的两种模式

02 强制指定工作流

Orchestrated Workflow

代表机制

定制技能（Skills）

工作方式

我们预先写好剧本和提示词，明确规定第一步必须调工具A，第二步必须调工具B，大模型只需要返回调用工具的 JSON 字符串即可。

特点

像流水线上的"熟练工人"，按照流程办事，相对稳定、可控，适合高频和标准化的任务，但是对于未知和开放性问题，可能无法处理好。

选择策略： 开放性问题用模式一（灵活但需监督），标准化任务用模式二（稳定且可控）。实际应用中，两者往往结合使用。

03

工具的两种主要形态

CLI工具 • API工具 • 形态对比与适用场景

工具的两种主要形态

CLI 工具

本地工具 • 命令行接口

定义

通过**命令行**与本地软件交互

典型例子

- 文件操作
- 代码执行
- Git 命令

> "像人一样打开软件、执行命令"

API 工具

网络服务 • 标准接口

定义

通过**标准接口**与外部系统对话

典型例子

- 搜索
- 天气查询
- 数据库访问

> "大模型学会了一种'请求格式'来调用外部服务"

形态选择： CLI 适合本地操作和开发任务，API 适合网络服务和外部系统集成。实际 Agent 往往同时使用两种形态。

04

CLI 工具详解

技术定义 • 执行流程 • CLI vs API 对比

CLI 工具：Agent 的"手和脚"

技术定义

CLI (Command Line Interface) 是通过输入文本命令来操作计算机或软件的一种方式。

对于大模型而言，CLI 就像是它的"手和脚"，让它可以直接在我们的电脑上"敲击键盘"。



为什么在 Agent 时代 CLI 工具变得极其流行?

万能的执行力

对程序员来说，**几乎所有的开发任务都可以通过 CLI 完成**：

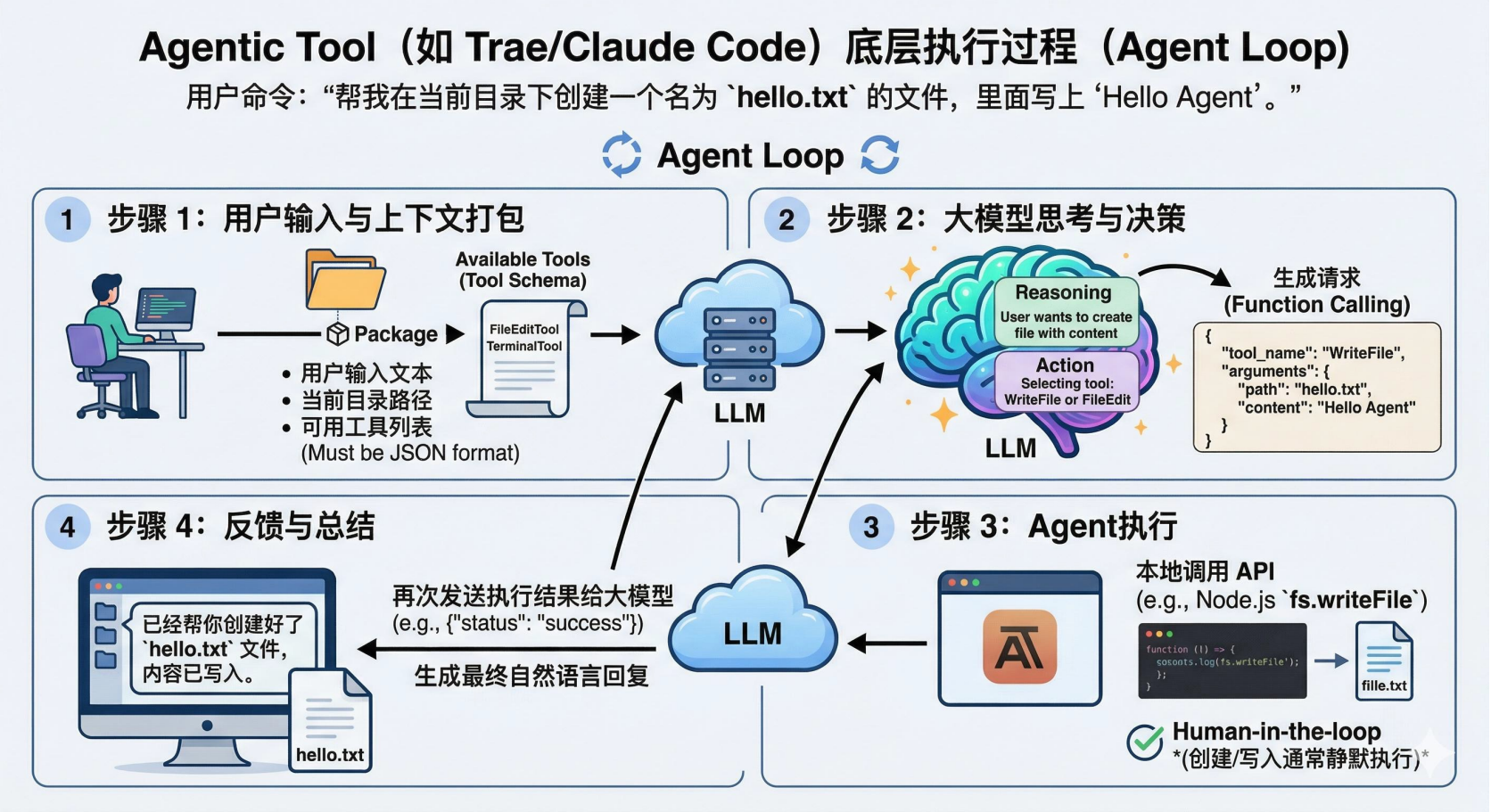
`npm test`	运行测试
`git commit`	版本控制
`pip install`	环境配置

Agent 的完美契合

像 **Trae** 这样的 Agentic IDE 可以直接在后台分配一个 **Terminal（终端）**：让大模型（逻辑上）**像真实程序员一样操作你的工作区**。

Agent 如何使用 CLI 工具：执行流程

场景： 用户输入：”帮我在当前目录下创建一个名为 hello.txt 的文件，里面写上’Hello Agent’。”



Agent 如何使用 CLI 工具：执行流程

1 用户输入与上下文打包

动作

你敲下回车键

后台发生的事情

Claude Code 将你的这句话，以及当前的 “上下文”一起打包发送给云端的Claude大模型，上下文包括：

- **当前所在的目录路径**
- **可用工具列表（Tool Schema）：**它会告诉模型，“我这里有几个工具你可以用，比如 FileEditTool（编辑文件）、TerminalTool（运行命令）等，并且规定了必须用 JSON 格式调用”。

2 大模型思考与决策（Thought &Action）

动作

云端的大模型接收到信息

后台发生的事情

- **思考（Reasoning）：**模型理解你的意图是创建文件并写入内容。
- **选择 工具：**它会在提供的工具列表中寻找。它可以选择用 Terminal 工具执行 `echo ‘Hello Agent’ > hello.txt`，也可以选择用专门的 File System 工具直接写入文件。通常，对于纯文本写入，模型会优先选择专门的**File System 写入工具**（如WriteFile 或 FileEdit）。

- **生成请求 (Function Calling)：** 模型输出一段 JSON，格式大致如下：

```
{
  "tool_name": "WriteFile",
  "arguments": {
    "path": "hello.txt",
    "content": "Hello Agent"
  }
}
```

Agent 如何使用 CLI 工具：执行流程

3 Agent 执行（Execution）

动作

Trae 获取到大模型的 JSON 请求

后台发生的事情

- Trae 不会把这串 JSON 打印给你看。
- 它在本地调用**操作系统的 API**（如 Node.js 的 `fs.writeFile`），按照 JSON 里的参数，在当前目录下真切地创建 `hello.txt` 并写入了内容。
- （注：如果是破坏性操作，比如删除文件，这里会触发 `Human-in-the-loop`，弹窗问你“允许执行吗？”。但创建/写入文件通常直接静默执行，或者给个小提示。）

4 反馈与总结（Observation & Output）

动作

本地执行完毕

后台发生的事情

- Trae 拿着执行结果（比如：`{"status": "success"}`），再次发送给云端的大模型，告诉它：“你刚才要求的工具调用成功了。”
- 模型收到成功反馈后，生成最终的自然语言回复：“已经帮你创建好了 hello.txt 文件，内容已写入。”

最终呈现

你在屏幕上看到了模型的回答，同时工作区里多出了那个文件。

Agent Loop: 这是一个循环过程——用户输入 → 模型决策 → 本地执行 → 反馈模型 → 生成回复。循环直到任务完成。

CLI vs API：以创建飞书文档为例

使用飞书 API

网络接口调用

HTTP POST 请求

```
curl -X POST  "https://open.feishu.cn/\
open-apis/docx/v1/documents" \ -H
"Authorization: Bearer  t-g10491X..." \ -H
"Content-Type:  application/json" \ -d
'{"title": "我的新文档"}'
```

- 需要查阅飞书开放平台文档
- 获取 `tenant_access_token`
- 构造复杂的 HTTP POST 请求
- 处理网络请求的回调和 JSON 解析

适用场景： 过程繁琐，适合在代码中做深度的系统集成

使用飞书 CLI

命令行工具

一行命令

```
lark-cli base +base-create --name "我的文档"
```

- **极简**：只需在终端输入一行命令
- **自动处理**：认证、Token 刷新、HTTP 请求
- **人类友好**：过程极简，适合人类直接使用
- **Agent 友好**：适合大模型通过调用命令行工具快速完成繁琐任务

适用场景： 适合快速完成任务和自动化流程

核心洞察： CLI 工具在底层自动帮你处理了繁琐的细节。过程极简，不仅适合人类直接使用，也非常适合让大模型（Agent）通过调用命令行工具来快速完成繁琐的任务。

05

Demo 演示

批阅作业并上传至飞书文档的完整工作流



Demo: 批阅作业并上传至飞书文档

场景描述

构建一个批阅作业并将结果上传至飞书文档的 SKILL

这是一个结合**本地文件处理**（读取作业）和**网络服务**（飞书文档创建）的完整工作流。

工作流程

1 读取作业

使用 CLI 工具

- 扫描作业文件
- 读取文件内容

2 批阅分析

大模型推理

- 分析答案正确性
- 生成批改意见

3 生成报告

格式化输出

- 统计分数
- 生成批阅文档

4 上传飞书

使用 API/CLI

- 创建飞书文档
- 写入批阅结果

技术栈

制作CLI脚本	LLM	文档生成	飞书 API
文件操作、统计汇总	推理引擎	格式化输出	文档上传

06

边界与安全

工具分类 • Human-in-the-loop • 安全框架

边界与安全：Human-in-the-loop

双刃剑效应

赋予大模型"手脚"是一把**双刃剑**。它可能会**误删文件**，或者**调用付费接口**造成损失。

只读操作

安全

- 查询天气
- 搜索网页
- 读取文件内容

不会产生副作用，通常直接执行

修改操作

危险

- 删除文件
- 执行终端命令（`rm -rf`）
- 提交代码

需要人工确认，触发 Human-in-the-loop

Human-in-the-loop（人类在环）

机制说明

对于**危险操作**，Agentic IDE（如 Trae）会**弹窗**，要求用户点击 `"Approve（批准）"` 后才会真正执行。

核心认知

人类始终是最终的决策者，AI 是副驾驶（Copilot）。

工具让 Agent 从"思考者"变为"行动者"

工具不是让大模型变聪明，而是让它从"知道"变成"做到"。

在 **Human-in-the-loop** 的安全框架下，Agent 才能真正释放其生产力价值。

思考
LLM 推理引擎

行动
CLI+API 工具

安全
人类在环机制

Agentic IDE 中的工具

End

2.氛围编程的关键实践 (7)

－Agentic IDE中的工具